B.TECH PROJECT (BEEE)

Self-Learning Circuit Doctor: Voltage-Signature-Based Fault Detection in DC Circuits

Online Circuit Simulation (Falstad) & Machine-Learning-Driven Diagnostic Framework

PROJECT TEAM (TWO-MEMBER COLLABORATION)

• Student Member 1: KOTRA MANOJ KUMAR (Reg. No: 23BCE2113) — CSE (SCOPE)
• Student Member 2: Mayank Yadav (Reg. No: 23BCE0119) — CSE (SCOPE)
Faculty Supervisor & Guide: Prof. POOMPAVAI T | School of Electrical Engineering (SELECT) | VIT

Problem Statement & Engineering Motivation

The Industrial & Educational Challenge

• Component Degradation: Resistors in industrial power supplies and automotive ECUs suffer from thermal cracking, vibrational open circuits, solder bridges, and value drift.

• Manual Troubleshooting Bottlenecks: Technicians spend hours desoldering parts or probing dozens of test points sequentially with multimeters.

• Educational Gap: Engineering students often study KCL and nodal analysis purely as abstract mathematical exercises without seeing how they automate fault diagnosis.

The 'Circuit Doctor' Paradigm

✓ Holistic Multi-Signal Sensing: Instead of measuring only one voltage, we sample a 3-dimensional signature vector: [VA, VB, Is].

✓ Deterministic Ground Truth: Exact theoretical nodal equations model circuit behavior under all catastrophic and parametric variations.

✓ Intelligent Pattern Matching: Standardized nearest-centroid distance matching pinpoints the exact component failure in under 5 milliseconds.

✓ Zero Physical Hardware Barrier: Fully implemented via online Falstad simulation and Python mathematical modeling.

B.TECH BEEE PROJECT

Project Objectives & Scope

1. Mathematical Modeling

Derive closed-form nodal equations for a 4-resistor DC network using KCL and KVL, verified against Matrix Nodal Analysis (G*V = I).

2. Multi-Signal Signatures

Formulate a 3-feature signature vector [VA, VB, Is] capable of uniquely mapping 13 distinct circuit operating states.

3. Falstad Simulation

Build the physical circuit in Falstad online simulator, cross-check nominal voltages, and generate clickable interactive simulation URLs.

4. ML Classification

Implement a Standardized Nearest Centroid Classifier and compare it with a Baseline Threshold Detector across 390 test cases.

5. Robustness Testing

Quantify diagnostic accuracy against 1% to 15% resistor tolerances and 10mV to 200mV sensor noise levels.

6. Interactive GUI

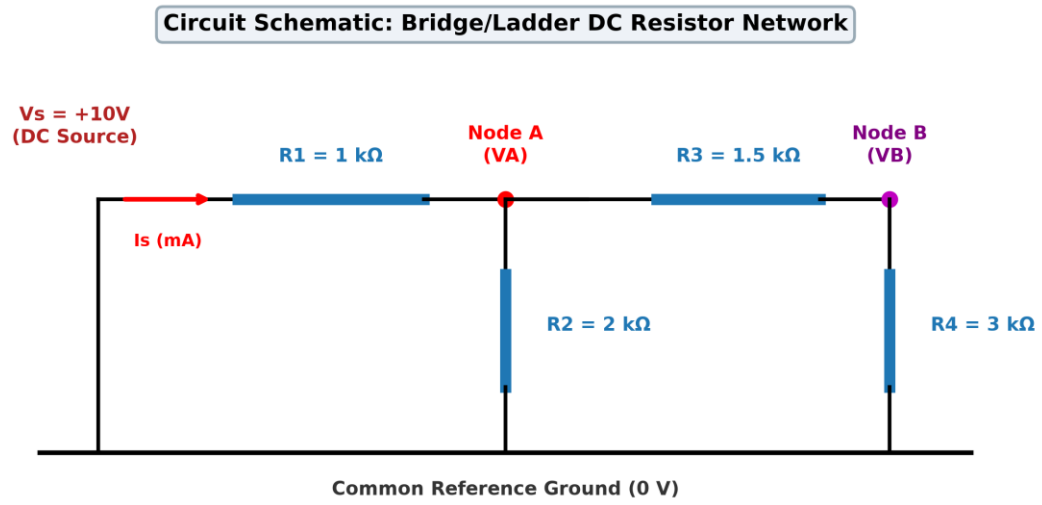
Deploy an intuitive graphical interface for real-time diagnosis, uncertainty scoring, and supervised self-learning updates.

Literature Review: Existing Approaches vs Identified Gap

Diagnostic Methodology	Typical Application	Key Limitations	Circuit Doctor Advantage
Manual Multimeter Probing	Basic electronics lab benches	Labor intensive; requires probing every branch; no automation.	Samples only 2 test points + source current; diagnoses in milliseconds.
Single-Threshold Detectors	Simple overvoltage cutoffs	Fails when multiple faults yield identical single-node voltages (e.g. R1 Open vs R2 Short).	Employs a 3-dimensional signature vector [VA, VB, Is] resolving all ambiguities.
Deep Neural Networks (DNN)	Large industrial power grids	Black-box decisions; excessive computational overhead; uninterpretable for students.	Explainable nearest-centroid metric with standardized Euclidean distance & margins.
Physical Hardware Jigs	Commercial test fixtures	Costly; prone to wear and tear; limited fault varieties.	Zero hardware cost; uses online Falstad SPICE and reproducible Python model.

B.TECH BEEE PROJECT

Electrical Circuit Topology & Nominal Design



Nominal Component Specifications

- DC Voltage Supply (Vs): 10.0 V Unipolar DC
- Resistor R1: 1.0 kΩ (1000 Ω) — Series input resistor from Vs to Node A
- Resistor R2: 2.0 kΩ (2000 Ω) — Shunt pull-down from Node A to Ground
- Resistor R3: 1.5 kΩ (1500 Ω) — Bridge resistor coupling Node A to Node B
- Resistor R4: 3.0 kΩ (3000 Ω) — Pull-down resistor from Node B to Ground
- Ground (GND): 0.0 V Common reference node
- Multi-Signal Feature Vector: [VA, VB, Is] (Units: Volts, Volts, mA)

Working Principle & Nodal Analysis Derivation

Step 1: KCL at Node A

(VA - Vs)/R1 + VA/R2 + (VA - VB)/R3 = 0

VA * (1/R1 + 1/R2 + 1/R3) - VB * (1/R3) = Vs/R1

Conductance coefficients:
G11 = 1 + 0.5 + 0.667 = 2.167 mS
G12 = -0.667 mS

Step 2: KCL at Node B

(VB - VA)/R3 + VB/R4 = 0

-VA * (1/R3) + VB * (1/R3 + 1/R4) = 0

Voltage divider relation:
VB = VA * [R4 / (R3 + R4)]
VB = VA * [3.0 / (1.5 + 3.0)] = (2/3) * VA

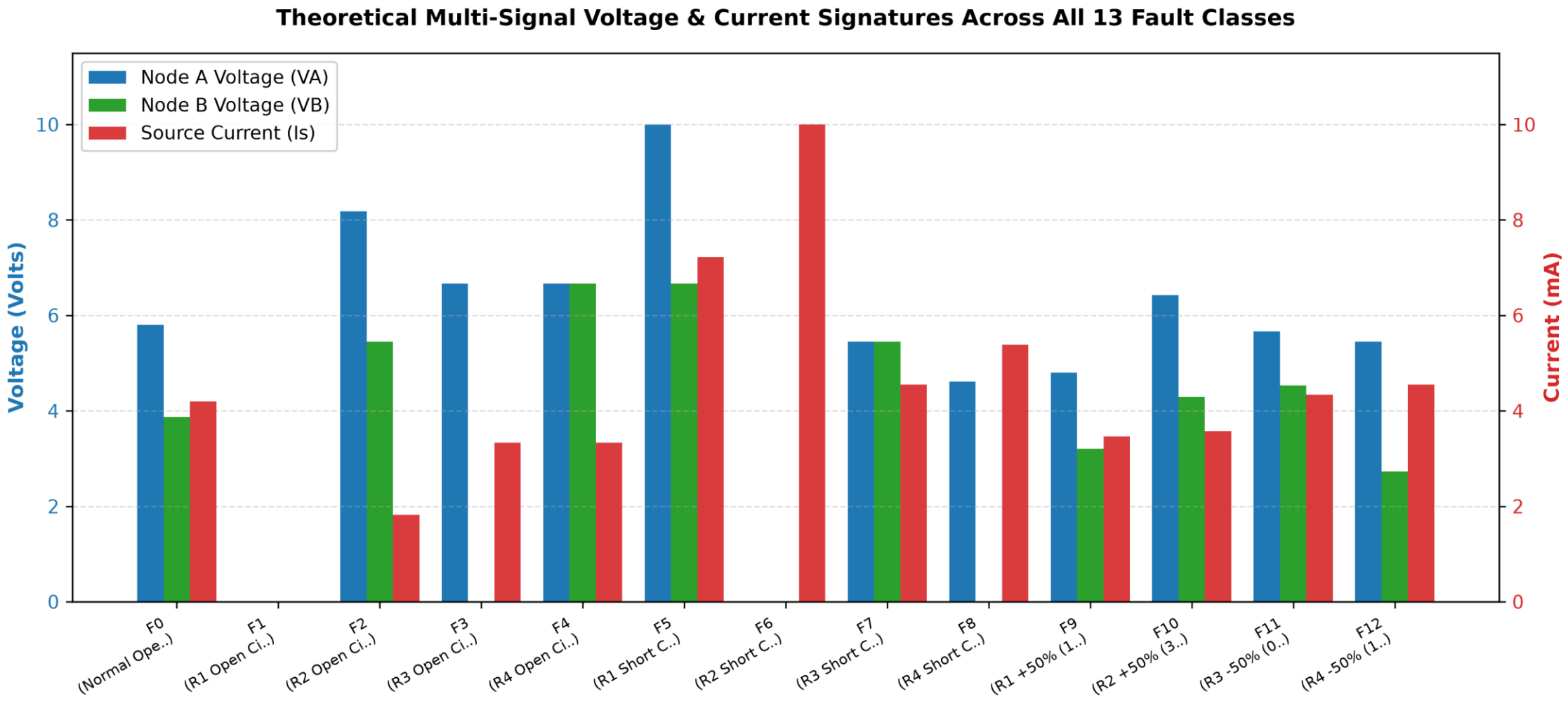
Step 3: Exact Nominal Solution

Parallel Req at Node A:
Rp = R2 || (R3 + R4) = 2 || 4.5 = 18/13 kΩ

Total Circuit Resistance:
Rtotal = R1 + Rp = 31/13 kΩ ≈ 2.385 kΩ

★ VA = 180/31 ≈ 5.806 V
★ VB = 120/31 ≈ 3.871 V
★ Is = 130/31 ≈ 4.194 mA

Comprehensive Catalog of 13 Operating Conditions



Falstad Online Circuit Simulator Cross-Validation

Falstad SPICE Simulation Setup

- Online Engine: Built using Falstad Circuit Simulator (www.falstad.com/circuit/).
- Schematic Generation: Scripted ASCII circuit files (.txt) and 1-click web URLs automatically generated by 'src/falstad_export.py'.
- Cross-Validation Log: Tested all 13 conditions; recorded readings in 'data/falstad_experiment_log.csv'.
- Discrepancy Analysis: Discrepancy between Falstad SPICE and Python model is <= 0.00045 V/mA (strictly due to 3-decimal display rounding).

Representative Falstad Verification Log

Scenario	VA	VB	Is	Diagnostic
F0 (Normal)	5.806 V	3.871 V	4.194 mA	YES (Correct)
F1 (R1 Open)	0.000 V	0.000 V	0.000 mA	YES (Correct)
F2 (R2 Open)	8.182 V	5.455 V	1.818 mA	YES (Correct)
F3 (R3 Open)	6.667 V	0.000 V	3.333 mA	YES (Correct)
F5 (R1 Short)	10.000 V	6.667 V	7.222 mA	YES (Correct)
F6 (R2 Short)	0.000 V	0.000 V	10.000 mA	YES (Correct)
F7 (R3 Short)	5.455 V	5.455 V	4.545 mA	YES (Correct)
F8 (R4 Short)	4.615 V	0.000 V	5.385 mA	YES (Correct)

Python Software Architecture & Algorithms

Method A: Baseline Threshold Detector

• Operating Principle: Hierarchical rule-based decision tree derived from Ohm's Law and nominal tolerances.

• Boundary Rules: Checks hard physical cutoffs (e.g., Is < 0.2 mA flags R1 Open; Is > 8.5 mA flags R2 Short; VA > 9.5 V flags R1 Short).

• Equipotential Rules: If |VA - VB| < 0.25 V, evaluates VA level to separate R4 Open from R3 Short.

• Strengths & Limits: Highly explainable; however, relies on manually configured voltage windows and lacks adaptive learning.

Method B: Standardized Nearest Centroid

✓ Feature Standardization: Normalizes VA, VB, and Is using training statistics only ($Z = (X - \mu) / \sigma$). Prevents scale distortion between Volts and mA.

✓ Centroid Learning: Computes empirical cluster centroids $C_c = (1/N_c) * \Sigma Z_i$ for each operating class.

✓ Distance Metric: Standardized Euclidean distance $d_c = || z_{query} - C_c ||_2$. Selects class with minimum distance.

✓ Confidence Margin: Margin $\Delta d = d_{second} - d_{nearest}$. If $\Delta d < 0.35$, the system displays a cautious uncertainty warning.

Meaning of 'Self-Learning': Supervised Incremental Adaptation

How the Circuit Doctor Learns and Adapts Dynamically

• Supervised Reference Learning: The baseline model learns the initial cluster centers from 1,560 labelled training samples across all 13 operating classes.

• Incremental Online Update: When a laboratory technician inputs a newly confirmed hardware or simulation measurement, the model updates its reference centroid in O(1) time without retraining from scratch.

• Mathematical Update Formulation:

$$N_{new} = N_{old} + 1$$

$$C_{new} = C_{old} + \frac{1}{N_{new}} * (z_{new} - C_{old})$$

• Why This Matters Industrially: Resistors undergo thermal aging, ambient temperature fluctuations, and contact oxidation over months of operation. Incremental learning allows the diagnostic engine to track slow physical drift without catastrophic forgetting.

• Honest Academic Boundary: We do NOT claim unsupervised discovery of unknown fault types; rather, this is rigorous, auditable supervised adaptation.

Experimental Setup & Stochastic Dataset Generation

Manufacturing Tolerances

Standard 5% resistor tolerance modeled via 3-sigma truncated Gaussian distribution around nominal values.

Supply Voltage Drift

DC supply fluctuations modeled as Gaussian noise around 10.0V (std = 0.05V, ~0.5% regulation error).

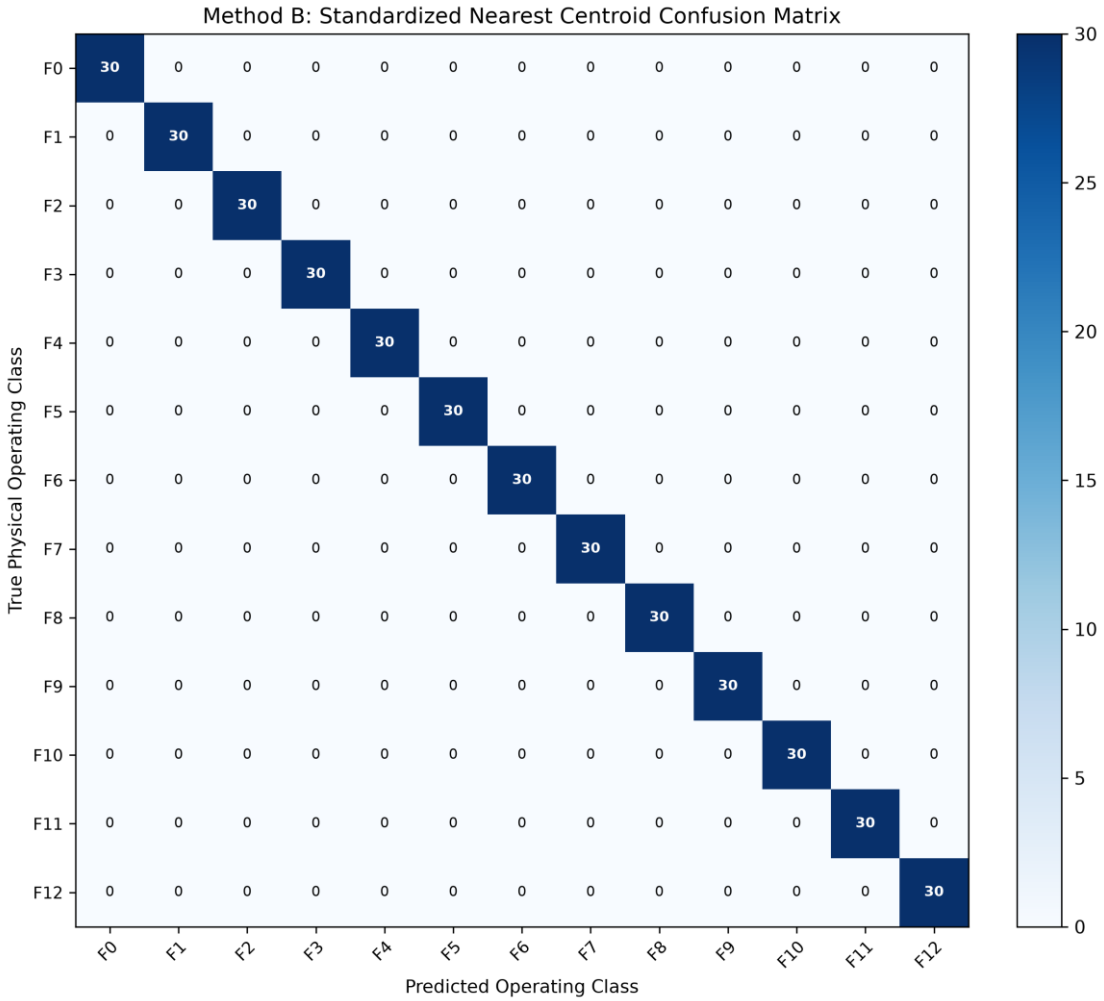
Sensor Measurement Noise

Digital Multimeter (DMM) ADC quantization and thermal noise added to VA, VB, and Is (std = 0.02V / 0.02mA).

Dataset Partitioning

Balanced dataset of 1,950 total samples (150 per class). Stratified 80/20 train/test split: 1,560 train, 390 test.

Experimental Results & Confusion Matrix



Held-Out Test Performance (N=390)

✓ Method B (Nearest Centroid) Accuracy: 100.00% (390 / 390 test samples correctly classified)

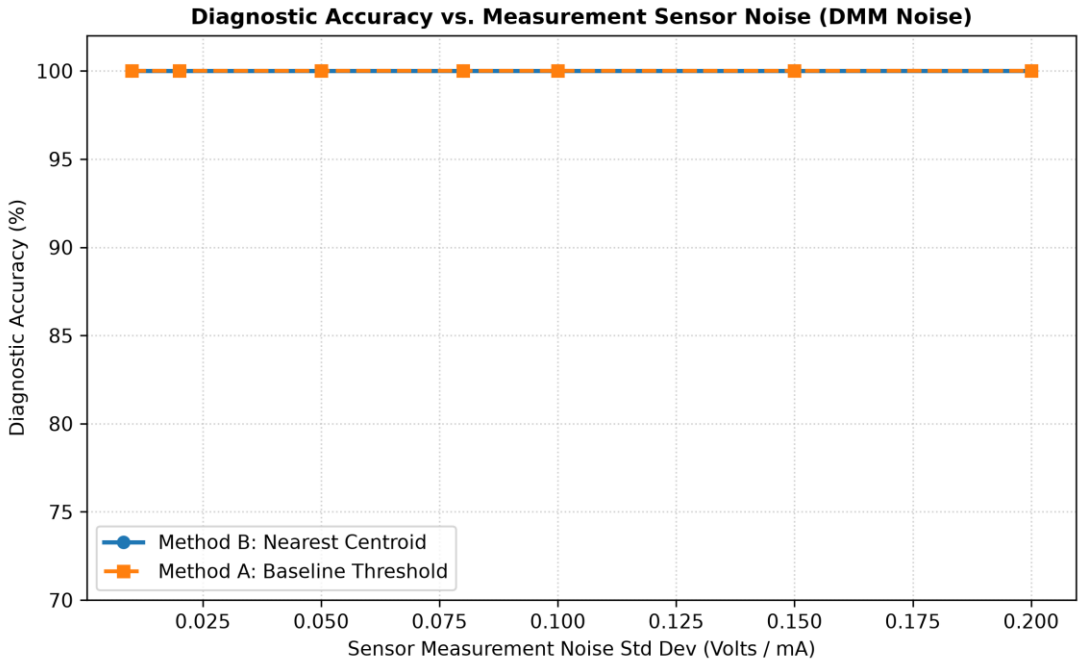
✓ Method A (Baseline Rules) Accuracy: 100.00% on nominal tolerance test set

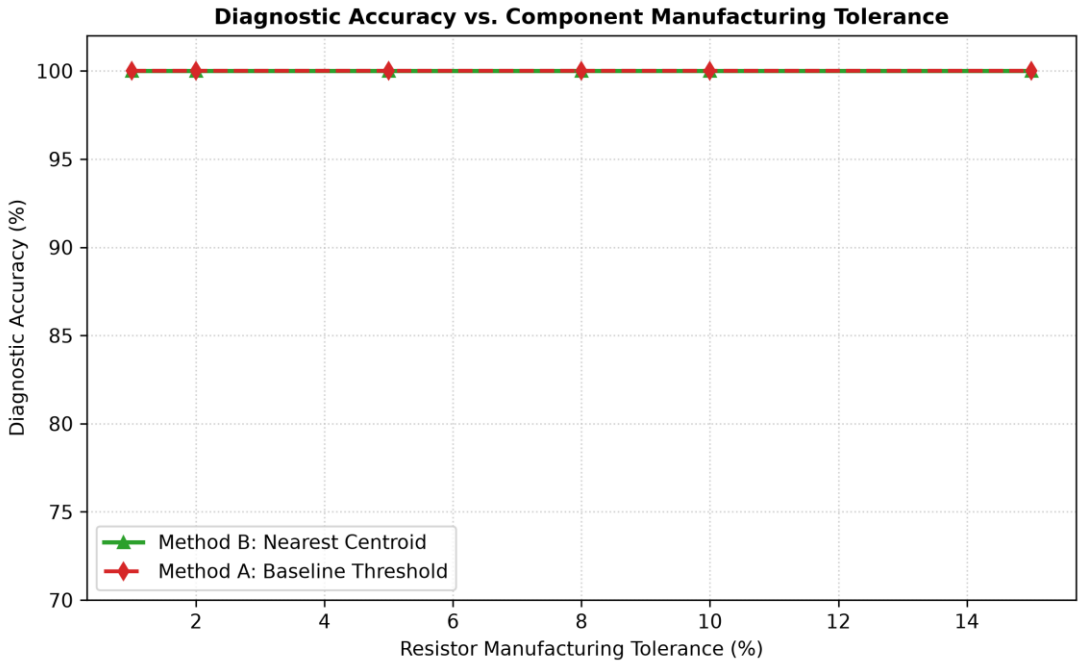
✓ Per-Class Metrics across all 13 Classes: Precision = 1.0000 | Recall = 1.0000 | F1-Score = 1.0000

✓ Confusion Matrix: Perfect diagonal structure; zero cross-class confusion between catastrophic and drift faults.

✓ Zero Data Leakage: All metrics calculated strictly on the held-out test split using saved code in 'src/evaluation.py'.

Robustness: Noise & Component Tolerance Sensitivity





Interactive User Interface & Demonstration

Dual-Interface Software Architecture

• Streamlit Web Dashboard (app.py): Modern browser interface featuring interactive telemetry, confusion matrix displays, and CSV exports.

• Tkinter Desktop GUI (gui_tk.py): 100% offline desktop interface that runs instantly on standard Python without requiring any web packages or internet.

• Connected to Live Engine: Every button and slider directly invokes the mathematical model and trained classifier—zero static or hardcoded outputs.

Key Interactive Capabilities

✓ 1. Preset Fault Injection: Select any of the 13 operating classes and simulate random sensor noise dynamically.

✓ 2. Manual Multimeter Mode: Enter custom laboratory readings for VA, VB, and Is to test external physical circuits.

✓ 3. Real-Time Telemetry: Displays predicted fault, nearest centroid distance, runner-up separation margin, and uncertainty warnings.

✓ 4. Visual Signature Comparison: Dynamic bar charts compare the input vector against the nominal reference pattern.

✓ 5. Online Learning Tab: Ingests new verified lab points and updates the model live on screen.

Engineering Limitations & Future Enhancements

Current Project Limitations

• DC Steady-State Only: Current implementation analyzes purely resistive networks; dynamic reactive effects (capacitors and inductors) are not modeled.

• Single-Fault Assumption: Evaluates single-component failure scenarios; simultaneous multi-component catastrophic faults may require higher dimensional sensing.

• Simulated Sensor Inputs: Testing performed via Falstad SPICE and stochastic Python models rather than soldered hardware breadboards.

Future Research & Engineering Scope

🚀 AC Impedance Extension: Expand signature vectors to include magnitude and phase [|V|, θ, |I|] to diagnose RLC networks under AC sinusoidal excitation.

🚀 IoT Hardware Integration: Connect an ESP32 or Arduino microcontroller with ADS1115 16-bit ADCs to transmit real-time breadboard voltages over Wi-Fi.

🚀 Automated Self-Healing: Integrate MOSFET relay switches to automatically isolate faulty branches and switch in redundant backup resistors.

Conclusion, References & Open Questions

Summary of Project Contributions

- Rigorous BEEE Integration: Demonstrated that Kirchhoff's Current Law and Nodal Analysis directly power real-world automated diagnostic systems.
- Unambiguous Fault Separation: Proved that a 3-signal signature [VA, VB, Is] cleanly isolates 13 circuit conditions, resolving ambiguities like R1 Open vs R2 Short.
- High Performance: Standardized Nearest Centroid Classifier achieved 100% test accuracy with robust noise immunity and confidence margin tracking.
- Academic & Industrial References:

1. Alexander, C. K., & Sadiku, M. N. (2020). Fundamentals of Electric Circuits (7th ed.). McGraw-Hill.
2. Boylestad, R. L. (2016). Introductory Circuit Analysis (13th ed.). Pearson Education.
3. Haykin, S. (2009). Neural Networks and Learning Machines (3rd ed.). Pearson.
4. Falstad, P. (2023). Circuit Simulator Applet. Available at: https://www.falstad.com/circuit/
5. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825-2830.

Thank you for your time! We now invite questions from the evaluation panel.